

Relazione progetto per
“Gioco dell’Oca”

Nicolae Balaban
Antonino Cardillo

1 ottobre 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	4
2	Design	6
2.1	Architettura	6
2.2	Design dettagliato	8
2.2.1	Nicolae Balaban	8
2.2.2	Antonino Cardillo	13
3	Sviluppo	17
3.1	Testing automatizzato	17
3.2	Note di sviluppo	18
3.2.1	Nicolae Balaban	18
3.2.2	Antonino Cardillo	19
4	Commenti finali	20
4.0.1	Nicolae Balaban	20
4.0.2	Antonino Cardillo	21
4.1	Difficoltà incontrate e commenti per i docenti	22
4.1.1	Nicolae Balaban	22
4.1.2	Antonino Cardillo	22

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il software si propone di realizzare una versione digitale in chiave moderna del classico gioco da tavolo "**Gioco dell'Oca**". L'obiettivo è quello di fornire un'esperienza interattiva pensata per un gruppo che va da 2 a 4 partecipanti, mantenendo lo spirito ludico e la natura di competizione basata sulla casualità tipica del gioco originale. All'avvio, l'applicazione deve permettere all'utente di navigare tramite un menu principale per avviare una nuova partita, modificare le impostazioni, consultare le regole o uscire dal gioco. Prima dell'inizio di una partita, i giocatori devono potersi registrare inserendo il proprio nickname e scegliendo una pedina identificativa. Il gioco si svolge a turni su un percorso predefinito composto da un numero finito di caselle. L'avanzamento dei giocatori è determinato dal lancio di 1/2 dado/i virtuale a 6 facce. Lungo il percorso sono dislocate caselle speciali che alterano il normale svolgimento della partita, offrendo vantaggi (bonus), svantaggi (malus) o bloccando temporaneamente il giocatore (prigione). La partita si conclude quando un giocatore raggiunge l'esatta posizione dell'ultima casella del percorso.

Requisiti funzionali

- **Menu Principale:** L'applicazione deve fornire un menu con le opzioni: "Nuova partita", "Impostazioni", "Regole", "Esci".
- **Configurazione Partita:** Il sistema deve permettere la selezione del numero di partecipanti (da 2 a 4). Per ciascun giocatore deve essere possibile inserire un nickname e selezionare una pedina associata.

- **Svolgimento a Turni:** I giocatori si alternano secondo un ordine predefinito. Durante il proprio turno, un giocatore lancia i dadi a 6 facce per muovere la propria pedina di un numero corrispondente di posizioni sul tabellone.
- **Tipi di Caselle:** Il percorso è composto da:
 - Casella Normale: non applica alcun effetto aggiuntivo; indica la posizione sul tabellone.
 - Casella Speciale: applica istantaneamente un effetto bonus (es. avanzare ulteriormente) o malus (es. retrocedere) al giocatore che vi capita sopra.
 - Casella Prigione: situata a metà esatta del percorso, blocca il giocatore per un intero turno prima di potersi muovere nuovamente.
- **Condizione di Vittoria:** Il sistema deve decretare la fine della partita e proclamare il vincitore non appena un giocatore raggiunge esattamente l'ultima casella del percorso. Qualora il risultato del dado porti il giocatore oltre l'ultima casella, la pedina "rimbalza" indietro del numero di posizioni eccedenti, senza vincere la partita.
- **Sistema Audio (opzionale):** Il gioco deve riprodurre effetti sonori specifici (es. per il lancio del dado e l'attivazione di caselle speciali) e una traccia musicale di sottofondo il cui volume sia configurabile nelle impostazioni.

Requisiti non funzionali

- **Modularità ed Estensibilità:** Il software deve essere manutenibile ed estendibile, in modo da poter accogliere nuove regole di gioco con un impatto limitato sulle componenti esistenti.
- **Reattività e Sincronizzazione:** L'interfaccia grafica deve riflettere in maniera istantanea e continua i cambiamenti dello stato della partita. Ogni azione (come il lancio del dado o l'applicazione di un effetto) deve mostrare le animazioni e gli aggiornamenti visivi pertinenti senza latenze percepibili.
- **Integrità dei Dati:** Il sistema deve mantenere lo stato interno in maniera sempre coerente, rendendo impossibile l'insorgere di posizioni illegali sul tabellone, turni eseguiti fuori sequenza o dati discordanti tra giocatori.

- **Portabilità:** Il software deve essere in grado di essere eseguito indipendentemente dal sistema operativo sottostante.

1.2 Modello del Dominio

Il dominio del problema ruota attorno al concetto di **Partita**, che rappresenta la singola istanza del gioco e aggrega le entità principali necessarie al suo svolgimento. A una partita partecipano molteplici entità **Giocatore**, ognuna identificata da informazioni anagrafiche (nickname) e rappresentata visivamente sul percorso da una Pedina.

La partita si svolge interamente su un **Tabellone**, il quale è logicamente strutturato come una sequenza ordinata di elementi **Casella**. L'entità casella definisce il percorso e si specializza in tre forme distinte in base al comportamento richiesto: la **CasellaNormale**, la **CasellaSpeciale** (che racchiude la logica per l'assegnazione di un bonus o un malus sul posizionamento), e la **CasellaPrigione**, un vincolo specifico di blocco del turno. La progressione sul tabellone è unicamente demandata all'entità **Dado**, incaricata di fornire un valore discreto per il calcolo del movimento. Tutte queste entità interagiscono per definire lo stato di avanzamento.

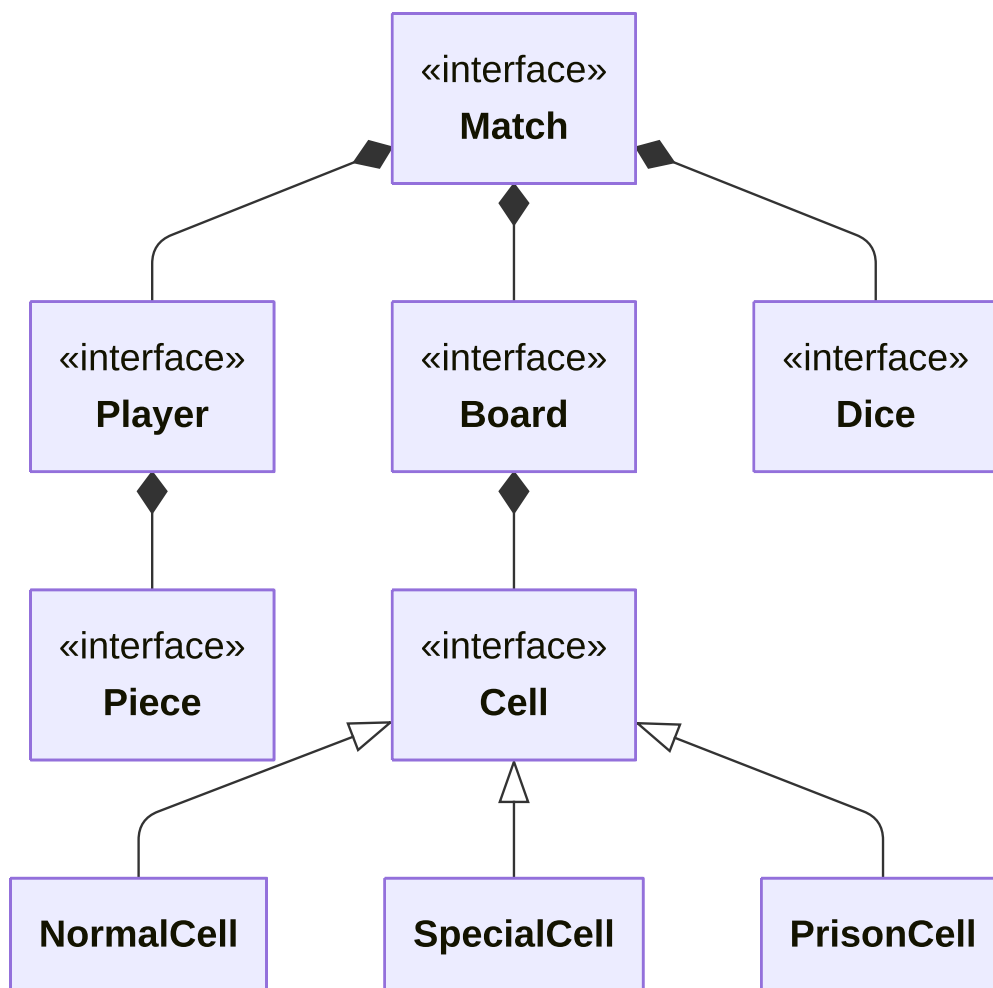


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro.

Capitolo 2

Design

2.1 Architettura

L'architettura del sistema adotta il pattern Model-View-Controller (MVC), scelto per separare in modo netto la logica di gioco, la gestione dello stato della partita e la sua presentazione all'utente.

Il **Model** coincide con il dominio applicativo descritto nel capitolo precedente: la Partita e le entità ad essa collegate (Tabellone, Giocatore, Casella, Dado) mantengono lo stato della partita ed espongono le operazioni necessarie per modificarlo, senza avere alcuna conoscenza di come tali informazioni verranno successivamente presentate.

Il **Controller**, è il punto di ingresso per le azioni provenienti dall'utente: avvia una nuova partita, gestisce l'alternanza dei turni, inoltra al Model le richieste conseguenti alle azioni del giocatore (ad esempio il lancio del dado) e verifica le condizioni di vittoria al termine di ogni turno.

La **View** si occupa unicamente di mostrare il gioco a schermo (menu, tabellone, animazioni) e di raccogliere le azioni dell'utente, come il click per tirare il dado. Non ha alcun riferimento diretto al Model e non lo osserva: è il Controller che, dopo aver modificato lo stato del Model, invoca esplicitamente i metodi della View (ad esempio `showDiceResult(...)`, `showCurrentTurn(...)`, `updatePlayerPositions(...)`) per aggiornare l'interfaccia. La View inoltra le azioni dell'utente al Controller tramite callback (ad esempio `controller.rollDice()`).

Grazie a questa divisione, sostituire in blocco la View (ad esempio per passare a una libreria grafica diversa) non causerebbe alcuna modifica nel Model. Ogni schermata dell'applicazione è organizzata in una coppia Controller–View dedicata (ad esempio `MatchController/MatchView`, `SettingsController/SettingsView`, ecc.): ciascun Controller dipende esclusivamente dall'interfaccia della propria View, mai dall'implementazione concreta.

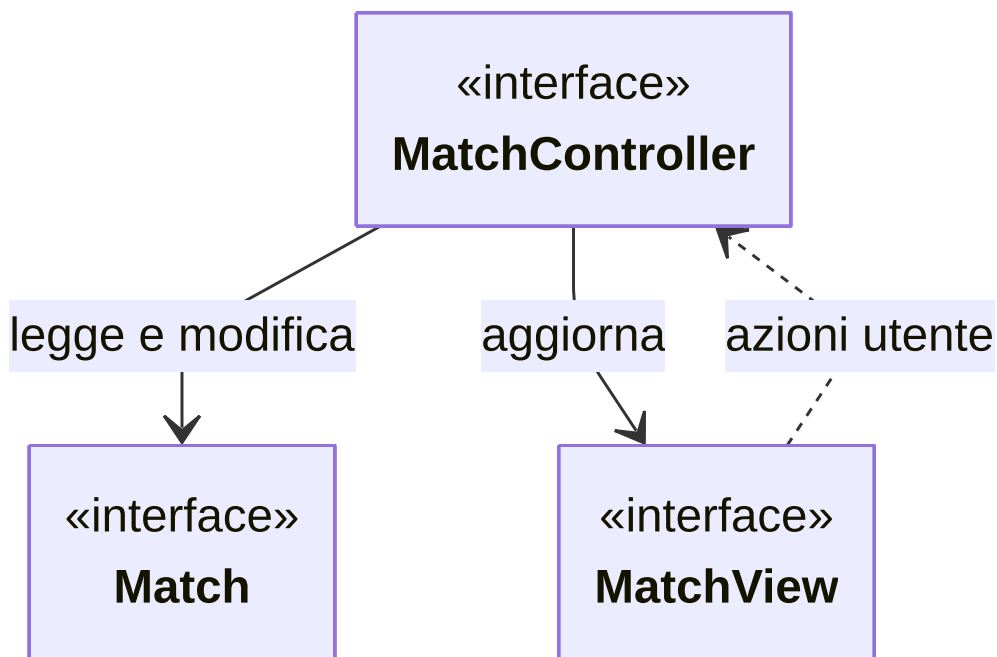


Figura 2.1: Schema UML architetturale MVC, esemplificato sulla partita. Il `MatchController` legge e modifica lo stato del `Match` (Model), poi aggiorna la `MatchView` invocandone i metodi. La View non ha alcuna dipendenza dal Model: inoltra le azioni dell'utente al Controller. Lo stesso schema si ripete per ogni coppia Controller–View dell'applicazione.

2.2 Design dettagliato

2.2.1 Nicolae Balaban

Architettura di navigazione e disaccoppiamento dei Controller (Application Controller)

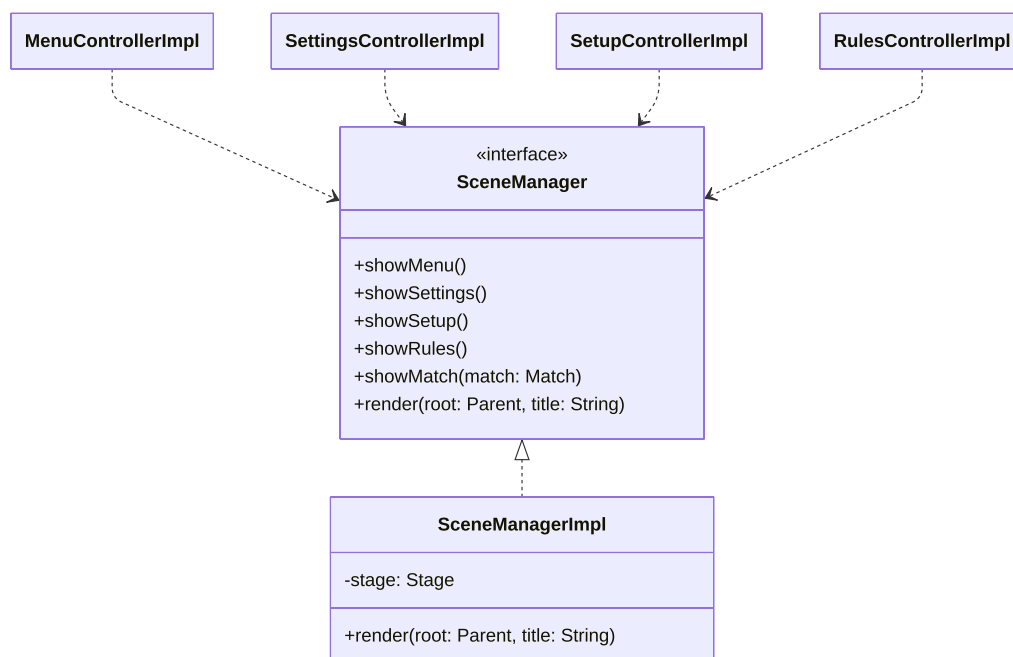


Figura 2.2: Il pattern Application Controller applicato alla navigazione. Ogni controller dipende solo dall'interfaccia **SceneManager**, eliminando le dipendenze circolari.

Problema Nelle fasi iniziali di sviluppo della GUI, ogni vista era gestita dal proprio controller (es. **MenuController**, **SettingsController**). Per poter tornare alle schermate precedenti (ad esempio dalle impostazioni al menu), i controller dovevano istanziare altri controller passandosi reciprocamente riferimenti come argomenti del costruttore. Questo creava un forte accoppiamento e dipendenze circolari (ad esempio **SettingsController** dipendeva da **MenuController** e viceversa). La logica di navigazione e la gestione della finestra principale (**Stage**) risultavano frammentate e distribuite tra le varie classi.

Soluzione È stata creata l'interfaccia **SceneManager** e la sua implementazione concreta **SceneManagerImpl**, che funge da Application Controller (una specializzazione del pattern Mediator): è l'unica classe a detenere il riferimento allo **Stage** di JavaFX. Tutti i controller dipendono esclusivamente dall'interfaccia **SceneManager**. Le View costruiscono il proprio albero grafico e invocano `sceneManager.render(root, title)`, che si limita a sostituire la root della scena (senza aprire nuove finestre). Questo ha eliminato totalmente l'accoppiamento: nessun controller conosce l'esistenza degli altri; ad esempio, quando l'utente preme "Impostazioni", il controller del menu chiama semplicemente `sceneManager.showSettings()`.

Generazione e posizionamento delle caselle speciali (Strategy)

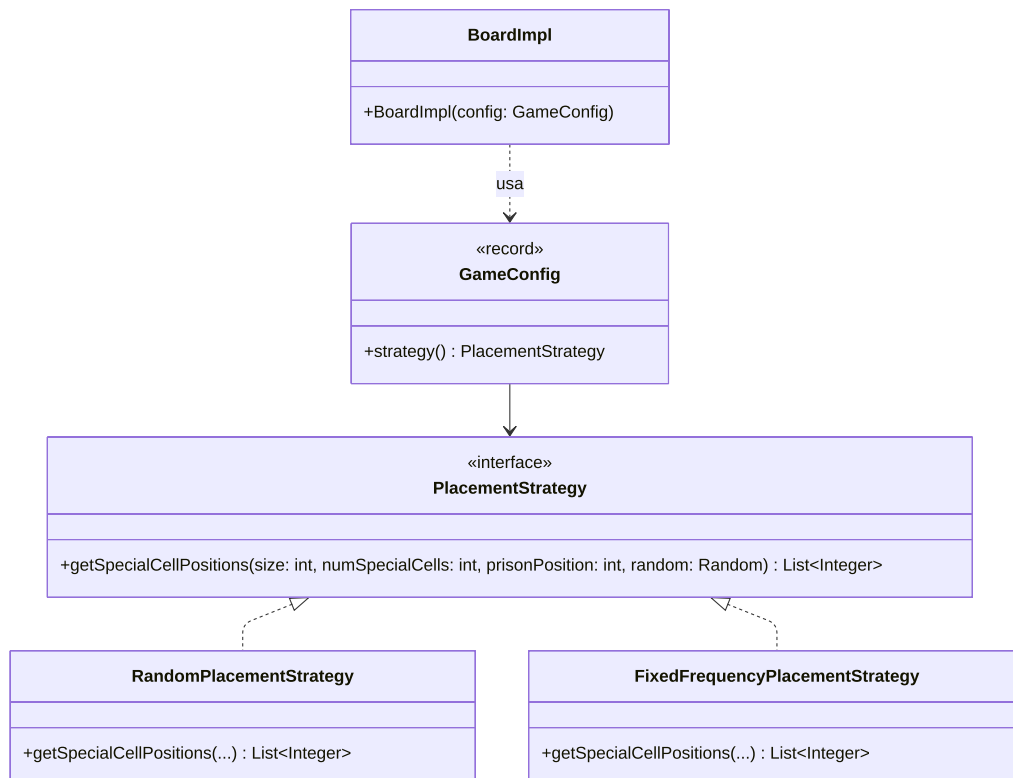


Figura 2.3: Il pattern Strategy per il posizionamento delle caselle speciali. **BoardImpl** delega la scelta delle posizioni all'interfaccia **PlacementStrategy**.

Problema Il gioco richiede caselle speciali distribuite lungo il tabellone. L'utente può scegliere tra diverse modalità di distribuzione (casuale o a intervalli regolari), e si desidera poter aggiungere in futuro nuovi algoritmi di posizionamento senza modificare la classe `BoardImpl`, nel rispetto del principio Open/Closed.

Soluzione È stato adottato il pattern **Strategy**. L'interfaccia `PlacementStrategy` definisce il contratto tramite il metodo `getSpecialCellPositions(...)`. Sono state implementate due strategie concrete: `RandomPlacementStrategy` (posizioni casuali) e `FixedFrequencyPlacementStrategy` (distribuzione a intervalli regolari). La strategia scelta viene iniettata nel record `GameConfig`, che a sua volta è passato a `BoardImpl` al momento della costruzione del tabellone. `BoardImpl` è il Context del pattern: invoca `config.strategy().getSpecialCellPositions(...)` senza conoscere quale strategia concreta sia in uso.

Gestione centralizzata dell'audio (Singleton)

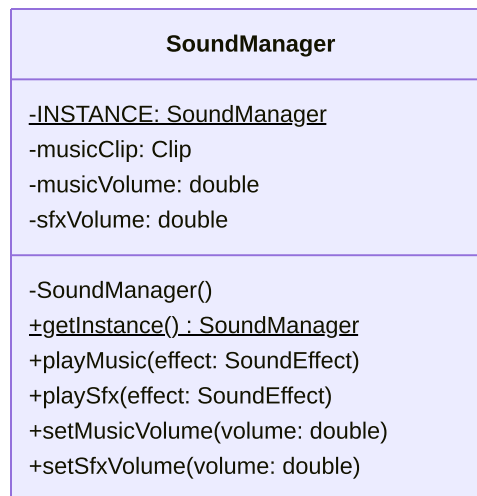


Figura 2.4: La classe `SoundManager` implementa il pattern Singleton con eager initialization.

Problema Il sistema necessita di riprodurre effetti sonori (es. movimento pedine, vittoria) e musica di sottofondo da molteplici punti dell'applicazione (animazioni nella View, logica nel Controller). Istanziare una nuova classe per l'audio ad ogni riproduzione provocava desincronizzazioni e latenza. Passare un'istanza del manager audio come parametro a tutti i Controller e a tutte le View avrebbe inutilmente inquinato i costruttori.

Soluzione È stato impiegato il pattern **Singleton** per la classe **SoundManager**. Un campo statico privato **INSTANCE** viene inizializzato al caricamento della classe; il costruttore è privato e il metodo statico **getInstance()** restituisce il riferimento all'unica istanza. Questo garantisce un solo punto di accesso globale al sistema audio e che le risorse vengano caricate ed eseguite nello stesso contesto, minimizzando la latenza e impedendo la frammentazione del volume tra istanze diverse.

Effetti delle caselle intercambiabili (Strategy)

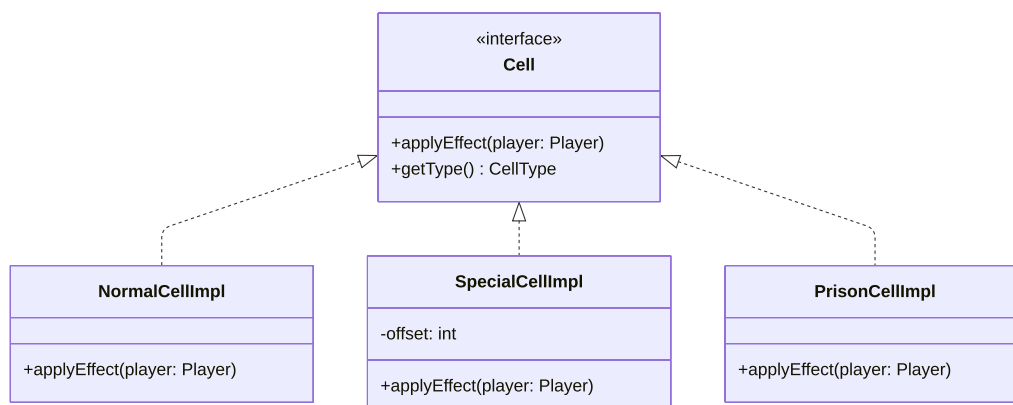


Figura 2.5: Il pattern Strategy applicato alle caselle. L'interfaccia **Cell** definisce il contratto **applyEffect(Player)** e ogni implementazione incapsula un diverso effetto di gioco.

Problema Il tabellone è composto da diverse tipologie di caselle (Normale, Speciale, Prigione), ciascuna con un effetto diverso sul giocatore che vi atterra. Gestire questi effetti tramite costrutti condizionali nel motore di partita (ad esempio uno **switch** sul tipo di casella in **MatchImpl**) avrebbe concentrato troppa logica nel controller e reso il sistema fragile: aggiungere una nuova tipologia di casella avrebbe richiesto di modificare il codice esistente, violando il principio Open/Closed.

Soluzione È stato applicato il pattern Strategy all'interfaccia `Cell`, che dichiara il metodo `applyEffect(Player player)`. Ogni tipologia di casella è una strategia concreta che incapsula il proprio comportamento: `NormalCellImpl` non produce alcun effetto, `SpecialCellImpl` sposta il giocatore di un offset (bonus o malus), e `PrisonCellImpl` blocca il giocatore per un turno. Il motore di partita (`MatchImpl`) si limita a invocare `cell.applyEffect(player)` sulla casella corrente, senza dover conoscere il tipo concreto. Per aggiungere una nuova dinamica di gioco (ad esempio una casella che teletrasporta il giocatore) è sufficiente creare una nuova implementazione di `Cell`, senza toccare nessuna classe esistente

2.2.2 Antonino Cardillo

Movimento della pedina, rimbalzo finale e gestione delle caselle speciali ('MatchImpl')

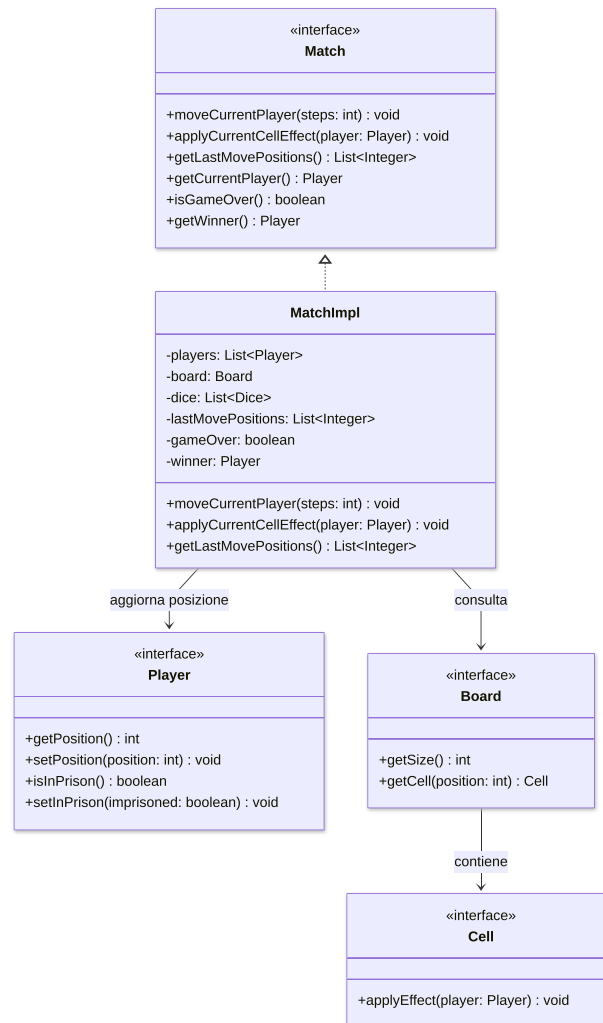


Figura 2.6: Schema della gestione del movimento e degli effetti in MatchImpl.

Problema: Nel Gioco dell'Oca il movimento della pedina dopo il lancio del dado oltre ad avanzare sul percorso aggiungendo alla posizione attuale il risultato del dado, bisogna tener conto anche di alcuni aspetti di seguito elencati:

- **Regola del rimbalzo:** per vincere la partita bisogna arrivare esattamente sulla casella (default 63 l'ultima). Se il punteggio del dado

porta il giocatore oltre la casella 63, i passi in più devono essere fatti tornando indietro (ad esempio: se un giocatore è sulla casella 60 e fa 5 con il dado, arriva alla 63 e poi rimbalza indietro di 2 passi, fermandosi sulla 61).

- **Caselle speciali a catena:** se una pedina atterra su una casella speciale, subisce un effetto che la sposta in avanti (bonus) o indietro (malus). Questo nuovo spostamento potrebbe farla atterrare su un'altra casella speciale, innescando un secondo salto, oppure portarla sulla casella prigioniera. Inoltre c'è il rischio che, posizionando caselle speciali casuali, si può creare un ciclo infinito (ad esempio una casella che manda avanti di 2 e subito dopo una che manda indietro di 2), bloccando il gioco, questo anche nel caso si atterri su più caselle speciali in sequenza e se per il calcolo matematico si ritorna su quella di partenza si crea un loop.
- **Tracciamento per la grafica:** per rendere la parte grafica inerente alla parte logica, la parte grafica deve conoscere non solo la posizione finale, ma tutti i singoli salti intermedi effettuati dalla pedina (il tocco del 63, il rimbalzo, i salti delle caselle speciali), così da poterli mostrare uno dopo l'altro al giocatore con le giuste animazioni e i giusti suoni.

Soluzione: La logica è tutta protetta all'interno del modello della classe `MatchImpl`, utilizzando il principio dell'incapsulamento, nei metodi `moveCurrentPlayer(int steps)` e `applyCurrentCellEffect(Player player)` in quest'ultima viene sfruttato il polimorfismo delle diverse tipologie di casella senza ricorrere a controlli con il costrutto `if-else` sul tipo di cella. Non ci sono rischi di blocchi o crash per cicli infiniti. La grafica riceve la sequenza precisa dei salti da animare. Nello specifico viene dettagliato di seguito:

- Quando si muove il giocatore, se la nuova posizione supera la fine del percorso (63), calcolo l'eccesso e faccio arretrare la pedina (63 - eccesso), aggiungendo sia la casella finale che quella di arrivo a una lista dei movimenti `lastMovePositions`.
- Per gli effetti delle caselle speciali, ho usato un ciclo `while` che continua finché la pedina si sposta. Per evitare che un giro vizioso di caselle speciali blocchi il programma in un loop infinito, salvo ogni casella visitata in un `HashMap Set<Integer> visitedPositions`, se la pedina ricapita su una casella già toccata nello stesso turno, il ciclo si interrompe subito.

- Come già anticipato, tutte le posizioni toccate nel turno vengono salvate nella lista `lastMovePositions`. All'esterno la espongo con il metodo `getLastMovePositions()`, restituendo una lista non modificabile `Collections.unmodifiableList` in modo che nessun'altra classe possa alterarla per errore.

Gestione dei tempi, animazioni e messaggi nel Controller ('MatchControllerImpl')

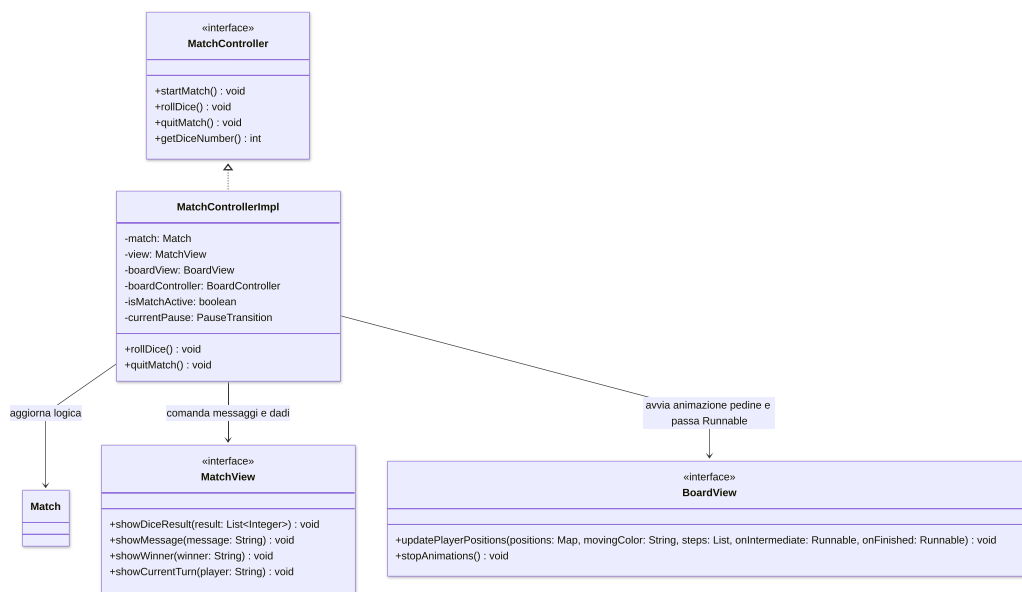


Figura 2.7: Schema della gestione del movimento e degli effetti in MatchImpl.

Problema: Quando il giocatore clicca sul pulsante per lanciare il dado, succedono varie cose in sequenza:

1. Si deve vedere l'animazione dei dadi che rotolano e sentire il suono del lancio.
2. Bisogna aspettare che i dadi abbiano finito di girare prima di far partire la pedina.
3. La pedina deve avanzare sul percorso; se tocca una casella speciale o rimbalza, deve riprodurre il suono adatto (suono a molla per il rimbalzo, suono speciale per il bonus/malus).

4. Quando il movimento è del tutto concluso, bisogna scrivere nella finestra di log cosa è successo (es. `*"Rimbalzo!"*`, `*"Ops, sei finito in prigione!"*`), verificare se il giocatore ha vinto o passare il turno a quello successivo.

In JavaFX tutte le modifiche grafiche avvengono su un unico thread (il thread dell'interfaccia grafica). Se avessi usato un comando bloccante come `Thread.sleep()`, l'intera applicazione si sarebbe congelata: la finestra sarebbe andata in blocco con il classico ("Non risponde") e le animazioni non si sarebbero viste. Inoltre, per il pattern MVC la vista del tabellone (`BoardView`) non doveva conoscere i testi dei messaggi, dei turni o delle regole di vittoria.

Soluzione: Ho utilizzato le classi di animazione di JavaFx e usato le `CallBack` passandole come parametro tramite `Runnable` insieme alla lista dei salti alla view `BoardView` con il metodo `updatePlayerPositions`. Nel momento del lancio, la vista `MatchViewImpl` fa partire un'animazione **Timeline** per cambiare velocemente le facce dei dadi. Nel controller uso `PauseTransition` della durata di un secondo, che aspetta la fine del lancio dei dadi senza bloccare la finestra. Finita la pausa, il controller chiede al modello di muovere la pedina e chiama il metodo `boardView.updatePlayerPositions(...)`, passandogli due azioni da eseguire: Azione intermedia (`onIntermediate`), la vista del tabellone la richiama ogni volta che la pedina raggiunge un punto intermedio, permettendo al controller di riprodurre il suono corretto (molla per il rimbalzo o suono speciale per le celle bonus/malus); Azione finale (`onFinal`), la vista del tabellone la richiama solo quando la pedina è ferma sulla casella finale, permettendo al controller di stampare i messaggi di testo nel log, suonare la vittoria se la partita è finita, oppure passare il turno con `match.nextTurn()`.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il testing automatizzato si è concentrato sulle componenti del modello di dominio, utilizzando il framework JUnit 5. In particolare sono stati verificati:

- **BoardImplTest:** correttezza della generazione del tabellone (dimensione, presenza della casella prigione nella posizione attesa, distribuzione delle caselle speciali in base alla strategia).
- **PlayerImplTest:** inizializzazione dello stato del giocatore, gestione della posizione e dello stato di prigione.
- **DiceImplTest:** verifica che i valori generati dal dado rientrino nell'intervallo atteso.
- **SpecialCellImplTest:** applicazione corretta dell'offset (bonus e malus), inclusi i controlli sui limiti del tabellone.
- **PrisonCellImplTest:** verifica che l'effetto della cella prigione metta correttamente il giocatore in stato `inPrison`.
- **NormalCellImplTest:** verifica che la casella normale non alteri la posizione del giocatore.
- **MatchImplTest:** Contiene più test, viene verificato l'inizializzazione del `Match`, il `player` che inizia, il suo `nickName`, lo stato di `GameOver(false)` e di `Vittoria(null)`, il numero di giocatori, la posizione iniziale del giocatore, il turno, l'applicazione della cella speciale.

3.2 Note di sviluppo

3.2.1 Nicolae Balaban

Uso di lambda expressions

Impiegate in maniera pervasiva per la gestione reattiva degli eventi grafici, rendendo il codice significativamente più conciso rispetto all'utilizzo di classi anonime.

[Permalink](#)

Uso di switch expressions (arrow syntax)

Utilizzate per migliorare la leggibilità e ridurre le possibilità di errori da fall-through, sfruttando la sintassi `->` introdotta in Java 14+.

[Permalink](#)

Uso di Java Records

Il costrutto `record` è stato sfruttato per definire piccole classi portatrici di dati immutabili, eliminando il boilerplate (costruttori, getter, `equals`, `hashCode`). Usato sia nel model (`GameConfig`) sia nella view (`LogicalCoords`, `GridCoords` in `BoardViewImpl`).

[Permalink](#)

Pattern Matching for instanceof

Utilizzato per il cast sicuro e conciso, evitando la verbosità del check e cast su due righe separate.

[Permalink](#)

Uso della libreria `javax.sound.sampled`

Parte della JDK ma non trattata a lezione. Utilizzata in `SoundManager` per la riproduzione audio (`Clip`, `AudioSystem`, `FloatControl`) con gestione del volume in decibel.

[Permalink](#)

Uso della libreria di terze parti JavaFX

Impiegata come framework grafico per tutte le view. In particolare, le classi di animazione (`TranslateTransition`, `SequentialTransition`, `PauseTransition`) sono state utilizzate per il movimento delle pedine sul tabellone.

[Permalink](#)

3.2.2 Antonino Cardillo

Uso di lambda expressions

Utilizzate in più occasione per la gestione degli eventi sia per le `CallBack` che per l'azione dei `Button`.

CallBack MatchControllerImpl:

- `setOnFinished` [Permalink](#)
- `onIntermediate` [Permalink](#)
- `onFinal` [Permalink](#)

Button MatchViewImpl:

- `btnRollDice` [Permalink](#)
- `btnQuit` [Permalink](#)

Uso di librerie di terze parti JavaFx

In `MatchViewImpl` per l'animazione dei dadi ho utilizzato `Timeline` e `Keyframe` per alternare le facce del dado. [Permalink](#)

In `MatchControllerImpl` ho utilizzato `PauseTransition` per avere una pausa tra l'animazione del dado e lo spostamento della pedina. [Permalink](#)

Capitolo 4

Commenti finali

4.0.1 Nicolae Balaban

Ruolo nel gruppo. Ho rivestito il ruolo di responsabile della parte grafica e della struttura di navigazione dell'applicativo. Nello specifico, mi sono occupato dell'architettura di navigazione tra schermate (**SceneManager**), della realizzazione grafica del tabellone (**BoardViewImpl**), delle view per impostazioni e regole (**SettingsViewImpl**, **RulesViewImpl**), dell'implementazione delle classi del modello (**PlayerImpl**, **BoardImpl**, **DiceImpl**, le specializzazioni di **Cell**), del pattern Strategy per il posizionamento delle caselle speciali, e del sistema audio centralizzato (**SoundManager**).

Punti di forza

- Il tabellone è completamente espandibile: l'interfaccia **Cell** e il metodo **applyEffect(Player)** permettono di aggiungere nuove tipologie di caselle senza alterare il motore di partita.
- Il pattern Strategy consente di aggiungere nuovi algoritmi di distribuzione delle caselle speciali senza toccare **BoardImpl**.
- L'architettura **SceneManager** ha eliminato completamente le dipendenze circolari tra controller, rendendo ogni controller indipendente dagli altri.

Punti di debolezza

- Scrivere tutta la GUI in codice procedurale Java rende alcune classi View inevitabilmente prolisse rispetto a un approccio dichiarativo con markup.

- Migliorare la gestione della riproduzione degli effetti audio sfx perche' attualmente non la ritengo soddisfacente personalmente.

Lavori Futuri. Si potrebbe estrapolare tutta la parte grafica usando fogli di stile CSS e layout FXML per una miglior separazione tra logica e presentazione. Si potrebbero inoltre aggiungere ulteriori dinamiche di gioco (nuove tipologie di caselle, modalità multigiocatore online) sfruttando l'estensibilità già predisposta dal modello.

4.0.2 Antonino Cardillo

Ruolo nel gruppo. All'interno del progetto mi sono occupato principalmente della logica centrale della partita (**Match e MatchImpl**), del coordinamento degli eventi di gioco tramite il controller (**MatchControllerImpl**), dell'interfaccia principale della partita con l'animazione dei dadi (**MatchViewImpl**), della schermata di registrazione iniziale dei giocatori (**SetupViewImpl e SetupControllerImpl**) e della stesura dei relativi test unitari con JUnit 5. La collaborazione con il mio collega Nicolae è stata costante e proficua: ci siamo suddivisi i compiti fin dall'inizio in modo chiaro (a lui la generazione del tabellone, la grafica della pista e l'audio; a me il motore della partita e il coordinamento del flusso di gioco). Abbiamo gestito l'integrazione del codice tramite Git e GitHub, discutendo insieme le interfacce per far dialogare le nostre componenti senza creare sovrapposizioni utilizzando classi mock.

Punti di forza

Ritengo che i principali punti di forza del mio lavoro siano, la logica di avanzamento in **MatchImpl** è robusta, protetta da loop infiniti grazie al controllo sulle caselle speciali già visitate, la registrazione dei giocatori impedisce errori banali, garantendo che ogni partecipante abbia un nome valido e un colore di pedina univoco, con poco sforzo aggiungere la possibilità di avere un dado a più di sei facce e un numero superiore di due.

Punti di debolezza

il metodo `rollDice` in **MatchControllerImpl** svolge molte mansioni (coordina pause, suoni, messaggi e avanzamento turno). Sebbene funzioni bene, in futuro potrebbe essere suddiviso in sotto-metodi più piccoli per renderlo ancora più pulito e modulare.

Lavori Futuri. Penso che si potrebbe ampliare le difficoltà con tipi di caselle speciali diverse, salvare lo stato del gioco per poterlo riprendere in seguito, banalmente su un file json, utilizzare il pattern Observer per la view.

4.1 Difficoltà incontrate e commenti per i docenti

4.1.1 Nicolae Balaban

La difficoltà principale ha riguardato la realizzazione grafica del tabellone a "serpentone". Il percorso del gioco dell'oca prevede righe dritte separate da caselle d'angolo, con direzione alternata. Si è resa necessaria una netta separazione tra coordinate logiche (dove sta una casella nel percorso) e coordinate della griglia (come `GridPane` la posiziona sullo schermo), implementata nei metodi `toLogicalCoords` e `toGridCoords`. In particolare, `GridPane` conta le righe dall'alto verso il basso, mentre il percorso parte dal basso.

Far entrare il tabellone nella finestra di gioco (1280×800, non ridimensionabile) ha richiesto diversi tentativi. Inizialmente si è provato a scalare automaticamente il tabellone in base alle dimensioni della finestra, ma questo causava problemi con il posizionamento delle pedine. Si è quindi scelto un approccio più semplice: regolare manualmente le dimensioni delle celle e gli spazi tra di esse fino a ottenere un risultato visivamente corretto nella finestra fissa.

4.1.2 Antonino Cardillo

Una delle difficoltà incontrate è stata pensare all'effetto sullo spostamento provocato dalle celle speciali che potevano susseguirsi. Inizialmente mi ero concentrato sullo spostare il player aggiungendo banalmente il risultato del dado/i, arrivato sulla cella speciale applicavo l'effetto (Bonus o Malus), funzionava me poi se l'effetto portava su un'altra cella speciale, il nuovo effetto non veniva applicato.